贞实科技
INDUSTRIAL NATIVE AI

指令
功能手册
V1.7.5

2026.05

文档版本

版本号	修订日期
1.3	2025 年 08 月 21 日
1.4	2025 年 08 月 30 日
1.5	2025 年 09 月 18 日
1.6	2025 年 10 月 14 日
1.7	2025 年 10 月 28 日
1.7.1	2025 年 12 月 15 日
1.7.2	2025 年 12 月 30 日
1.7.3	2026 年 01 月 16 日
1.7.4	2026 年 01 月 22 日
1.7.5	2026 年 05 月 14 日

目录

文档版本 I
一、常用变量类型 1
1. load 变量 1
2. jointtarget 关节空间参数 1
3. robottarget 笛卡尔空间参数 2
4. speed 速度参数 2
5. zone 转弯半径 3
6. tool 工具参数 4
7. wobj 基坐标系参数 5
8.其他变量类型 5
二、基础运动指令 6
1. Var 定义变量 6
1.Enable 上使能 7
2.Disable 下使能 8
3.Recover 状态同步 9
4.Start 开始 9
5.Stop 停止 10
6.Pause 暂停 10
7.Sleep 系统休眠等待 11
8.MoveAbsJ 关节运动 11
9.JogJoint 关节点动 12
10.JogCartesian 笛卡尔点动（位置+姿态） 13
11.JogC 单步笛卡尔点动 14
三、在线规划指令 15
1.SpeedJ 在线关节速度控制 15
2.SpeedL 笛卡尔空间在线速度控制 16
3.JogAnyJ 在线关节角度规划 17
4.JogAnyC 在线笛卡尔位置规划 18
5.JogAnyCpos 在线笛卡尔位置规划 19
6.SetEgmOffset 在线偏移控制 21

四、常用系统指令 22
1.Clear 错误清理 22
2.SetTool 设置工具 22
3.SetWobj 设置基坐标系 24
4.SetUsingSP 最优规划求解 25
5.SubLoop 多模型控制优化指令 25
五、高级规划指令 28
1.MoveBlend 笛卡尔空间下轨迹融合(局部) 28
2.MoveBlendScurve 笛卡尔空间下轨迹融合（匀速） 32
3.MoveBlendOptimalpos 笛卡尔空间下轨迹融合（最大速） 34
4.MoveS 笛卡尔空间下轨迹融合（全程） 36
5.ServoSeries 关节空间下轨迹融合指令 38
6.MoveSeriesToppJ 关节空间下轨迹融合（最大速） 39
六、力控指令 40
1.六维力传感器概述 40
2.ForcePositionHybridControl 笛卡尔空间下力-位混合控制 41
3.DragInCST 电流环下机器人拖动控制 43
七、IO 模块指令 44
1.GetDI 读取输入电平状态 44
2.SetDO 设置输出电平状态 44
3.DOPulse 输出数字脉冲信号 45
4.AddIO 注册新的 I/O 信号输出口 46
八、底盘导航指令 47
1.ModbusTCP 运动指令 47

一、常用变量类型

1. load 变量

➢ **变量说明**

该结构体包含一个刚体在某一个坐标系下的十个描述质量分布特性的 double 型参数：mass 质量，cogx 质心在 x 方向的偏移量，cogy 质心在 y 方向的偏移量，cogz 质心在 z 方向的偏移量，i 表示惯性张量矩阵参数(ixx,iyy,izz,ixy,ixz,iyz)。（质量单位：千克 kg；偏移量单位：米 m）

对于机器人来说，该参数可用于描述机器人末端负载参数特性，坐标系一般默认为末端法兰盘坐标系

➢ **使用示例**

Var --name=piece --type=load --value={mass, mass*cogx, mass*cogy,mass*cogz,0,0,0,0,0,0}

2. jointtarget 关节空间参数

➢ **变量说明**

jointtarget 有 2 种调用方式，一种为 Var 声明后，运动指令即可通过--jointtarget_var 调用,一种为运动指令添加--jointtarget_value={ J0,J1,J2,J3,J4,J5,J6,J7,J8,J9}参数调用。

该结构体共有十个 double 型参数（J0~J9），存储机器人目标关节角度（单位是弧度）和外部轴位置（单位是米或弧度）。会从该十个数依次匹配目标关节角度与外部轴位置。目标关节角度与外部轴总数不得超过十个。

➢ **使用示例**

Var --name=home --type=jointtarget --value={J0,J1,J2,J3,J4,J5,J6,J7,J8,J9} //Var 声明后，运动指令即可通过 --jointtarget_var 调用

3. robottarget 笛卡尔空间参数

➢ **变量说明**

robottarget 有 2 种调用方式，一种方式为 Var 声明后，运动指令即可通过--robottarget_var 调用，一种方式为运动指令中添加--robottarget_value={ x,y,z,q1,q2,q3,q4}参数调用。

该结构体储笛卡尔空间的目标位置及姿态（double），包含 x, y, z, q1, q2, q3, q4，适用于 MoveJ, MoveL，MoveC 指令的，位置单位为米。（q1,q2,q3,q4 分别对应 x,y,z,w）

➢ **使用示例**

Var --name=p1 --type=robottarget --value={x,y,z,q1,q2,q3,q4}

4. speed 速度参数

➢ **变量说明**

speed 有 2 种调用方式，一种是运动指令中加 --speed=v10，一种是运动指令中加 --speed_value=0.1，(建议在笛卡尔空间运动指令中使用)单位为 m/s。

该结构体包含 5 个 double 型成员变量，per 关节速度百分比，适用于 MoveAbsJ 和 MoveJ。tcp 线速度，eri 空间旋转速度，exj_l 外部轴线速度，exj_r 外部轴角速度。示例：speed v1={40 300 100 200 1000}

➢ **使用示例**

Var --name=v1 --type=speed --value={per,tcp,ori,exj_r,exj_l}

在 xml 文件的 model-Variable 节点下，有一些已定义好的 speed 结构体，例如 v100={0.1,0.1,3.4906585,0,0}，其中第一个 0.1 表示关节 max_vel 的 10%。

5. zone 转弯半径

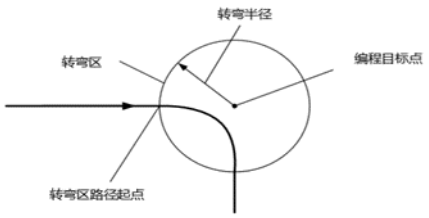
➢ 变量说明

该结构体变量用于定义某一个运动如何结束或者说定义两条运动轨迹之间转弯区的大小。该结构体包含两个 double 型变量，distance 为笛卡尔空间的转弯区大小，适用于 MoveLL 指令，单位是毫米；percent 为转弯区所占运动时间的百分比，适用于 MoveJ、MoveL、MoveC 等运动指令直接的融合

转弯区的距离大小不能超过前后两条待融合路径长度的一半，如果超过，系统会自动将转弯区长度缩小到前后两条路径中长度较小的一半；转弯区的时间不能超过前后两条待融合运动时间的一半，如果超过，系统会自动将转弯区时间缩小到前后两条路径中时间较小的一半。使用转弯区可以避免机器人频繁启停，显著减少节拍时间。

➢ 使用示例

Var --name=z100 --type=zone --value={dis,per}



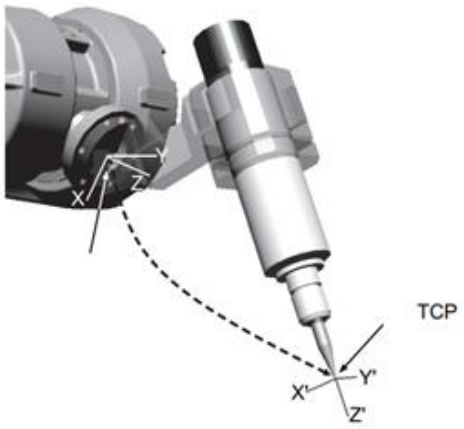
6. tool 工具参数

➢ 变量说明

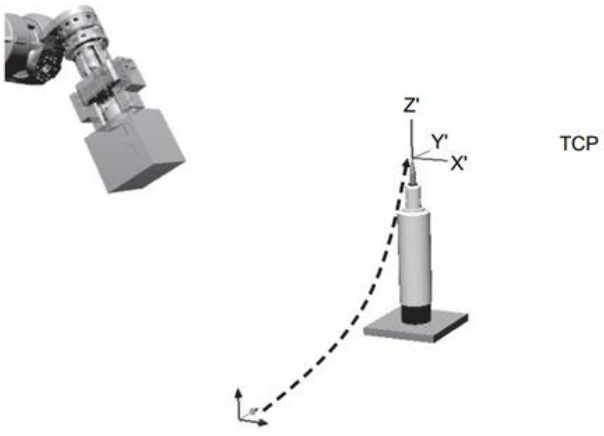
该结构体记录工具参数，包含机器人所用工具的 TCP、姿态，具体包含以下三个子结构体： pose 类型，包含 TCP 相对于机器人末端法兰盘坐标系的 x,y,z 三个方向的偏移量，单位为毫米，以及工具坐标系相对于法兰坐标系的姿态偏移量，用四元数表示。pos 结构体如下：pose pose1={x,y,z,q1, q2, q3, q4}。

➢ 使用示例

Var --name=tool --type=tool --value={true,x,y,z,q1,q2,q3,q4}



如果使用固定工具，则相对于世界坐标系来定义工具坐标系。



xx1100000518

Figure 3.4: 固定工具

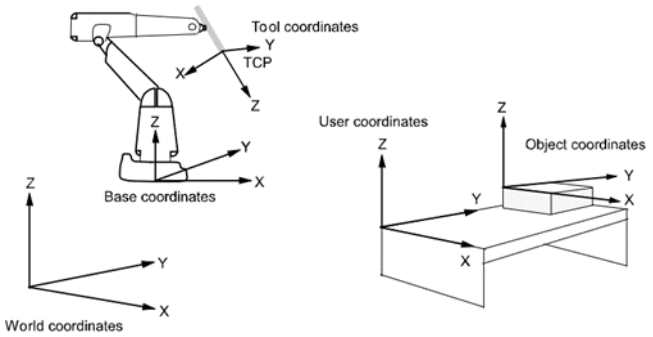
7. wobj 基坐标系参数

➢ **变量说明**

该结构体所有运动指令中使用的位置都是在工件坐标系下定义的（如果没有指定工件坐标系，则默认 在世界坐标系下定义，世界坐标系可以被看做是 wobj0）。

➢ **使用示例**

Var --name=wobj2 --type=wobj --value ={x, y, z, q1, q2, q3, q4}



8.其他变量类型

➢ **变量类型** Number ➢ **变量说明** 双精度浮点数

➢ **变量类型** Matrix ➢ **变量说明** 数值型数组

➢ **变量类型** String ➢ **变量说明** 字符串

二、基础运动指令

1. Var 定义变量

➢ 指令用法	定义变量

➢ 指令参数	数据类型	含义
--name	String	变量名
--type	String	变量类型
--value	Number/Matrix	变量值
--degree_unit	Number	输入类型
--clear	None	清除所有已定义变量

➢ 指令说明

1）当其他指令参数需要调用这些变量时，可通过 --type_var 的方式调用；

2）--type 支持的变量类型有 Number,Matrix,String,pos,orient,pose,speed,robottarget,zone,jointtarget,tool,wobj;

3）--value 可以是单个值，也可以是用大括号包含的矩阵;

4）var 为临时变量，重启后不保存，使用方式为 _var ,如： --jointtarget_var 、--robottarget_var 、--speed_var 等等（见运动指令中的使用示例）。

5）在定义关节变量（jointtarget）时，可以通过 degree_unit=1 来选择 value 单位为角度，degree_unit=0 则 value 单位为弧度。

6）变量名称不支持纯数字，需要以英文字母开头

➢ 指令示例

```
Var --name=pi --type=Number --value=3.14
Var --name=pe --type=Matrix --value={0.3,0.2,0.1,0,0,0}
Var --name=z2 --type=zone --value={0.1,0.1}
Var --type=Matrix --name=m0 --value={1,1,1,0.1,0.1,0.1}
Var --type=robottarget --name=pq --value={0,0,0,0,0,0,0}
Var --type=jointtarget --name=j0 --value={0.1,-0.1,-0.1,-0.1,0.1,0.4,0,0,0,0} --degree_unit=1
Var --type=tool --name=toolPP --value={0.1,0,0,0,0,0,1}
Var --type=load --name=load1 --value={0,0,0,0,0,0,0,0,0,0}
Var --clear  //清除所有 var 变量
```

1.Enable 上使能

➢ 指令用法	使能所有的伺服电机设备或者某一个伺服电机设备

➢ 指令参数	数据类型	含义
-- limit_time	Number	容许的最大等待时间,单位为毫秒。 当在该时间内未能使能成功，则系统会 抛出异常，错误等级为 ERROR。
--all/--motion_id	Number	--all 表示对所有电机使能；--motion_id=i 时表示对第 i+1 个电机使能。

➢ **指令说明**

➢ **指令示例**

例 1

Enable：相当于 Enable --all --limit_time=5000，或 Enable -a --limit_time //功能为使能全部电机，使能超时时间为 5000ms。

例 2

```
Var --type=load --name=load1 --value={0,0,0,0,0,0,0,0,0,0}
Enable --motion_id=0 --limit_time=4000 //功能为使能第一个电机，使能超时时间为4000ms。
```

2.Disable 下使能

➢ **指令用法**	下使能所有的伺服电机设备或者某一个伺服电机设备	
➢ **指令参数**	数据类型	含义
-- limit_time	Number	容许的最大等待时间,单位为毫秒。 当在该时间内未能使能成功，则系统会 抛出异常，错误等级为 ERROR。
--all/--motion_id	Number	--all 表示对所有电机使能；--motion_id=i 时表示对第 i+1 个电机使能。

➢ **指令说明**

➢ **指令示例**

例 1

Disable：相当于 Disable --all --limit_time=5000，或 Disable -a --limit_time=5000 //功能为使能全部电机，使能超时时间为 5000ms。

例 2

```
Var --type=load --name=load1 --value={0,0,0,0,0,0,0,0,0,0}
Disable --motion_id=0 --limit_time=4000 //功能为使能第一个电机，使能超时时间为4000ms。
```

3.Recover 状态同步

➢ 指令用法	同步模型状态与实际被控对象的状态
➢ 指令参数	无
➢ 指令说明	从 CST 模式切换到 CSP 模式前，要使用该指令，用于同步实际模型状态，不然无法通过 Show 指令获取机器人当前实际状态。

➢ **指令示例**

无

4.Start 开始

➢ 指令用法	将被暂停或停止的运动重新启动

➢ **指令参数**

数据类型	含义
--last_count	Number	运动重启到位所需的控制周期数目

➢ 指令说明	无

➢ **指令示例**

例 1

Start：相当于 Start --last_count=10 //花费 10 个控制周期完成运动重启。

Start --last_count=5 //花费 5 个控制周期完成运动重启。

5.Stop 停止

➢ 指令用法	停止运动，并跳出该指令。该指令后面的缓冲指令均不执行

➢ 指令参数	数据类型	含义
--last_count	Number	完全停止运动所需的控制周期数目

➢ 指令说明	1-可用于紧急停止机器人当前运动，可使用默认参数； 2-Stop 与 Start 一一对应； 3-多条指令阻塞调用时，一个 Stop 只能终止一条指令。

➢ 指令示例

例 1

```
Stop：相当于 Stop --last_count=10   //花费 10 个控制周期完全停止运动。
Stop --last_count=5   //花费 5 个控制周期完全停止运动。
```

6.Pause 暂停

➢ 指令用法	暂停运动，不跳出该指令。Start 后该指令可继续执行。该指令后面的缓冲指令会继续执行

➢ 指令参数	数据类型	含义
--last_count	Number	完全停止运动所需的控制周期数目

➢ 指令说明

➢ 指令示例

例 1

```
Pause：相当于 Pause --last_count=10   //花费 10 个控制周期完全暂停运动。
Pause --last_count=5   //花费 5 个控制周期完全暂停运动。
```

7.Sleep 系统休眠等待

➢ 指令用法	程序执行时，系统线程休眠，暂停后续指令执行，等待 指定时长后继续运行下一条指令。休眠期间系统不占用 CPU 资源，仅挂起等待。

➢ 指令参数	数据类型	含义
--count	Number	休眠时长

➢ 指令说明

➢ 指令示例

例 1

```
Sleep --count=1000     //延时等待指令
```

8.MoveAbsJ 关节运动

➢ 指令用法	关节空间运动

➢ 指令参数	数据类型	含义
--jointtarget/ --jointtarget_var/ --jointtarget_value/	jointtarget	目标关节角度变量（详情见 jointtarget 关节空间参数）
--speed/ --speed_var	speed	速度变量（详情见 speed 速度参数）
--zone/--zone_var	zone	轨迹融合变量

➢ 指令说明	无

➢ 指令示例

例 1

```
Var --type=jointtarget --name=j0 --value={0.1,0.2,0.3,0.4,0.5,0,0,0,0,0}
MoveAbsJ --jointtarget_var=j0：其他均取默认值
```

例 2

```
MoveAbsJ --jointtarget_value={0.1,0.2,0.3,0.4,0.5,0,0,0,0,0} –speed=v30
```

9.JogJoint 关节点动

➢ 指令用法		关节空间指定轴 Jog 运动
➢ 指令参数	数据类型	含义
--increase_count	Number	持续时间
--speed --speed_var	speed	关节速度，单位为 rad/s （详情见 speed 速度参数）
--motion_id	Number	关节索引，--motion_id=i 时表示是第 i+1 个电机
--direction	Number	运动方向：1-正方向运动，-1-负方向运动
--margin	Number	安全余量
➢ 指令说明		默认参数：--stop

➢ **指令示例**

例 1

```
JogJoint --speed=v100 --motion_id=0 --increase_count=50 --direction=1
JogJoint --speed=v100 --motion_id=1 --increase_count=50 --direction=-1
JogJoint --speed=v100 --motion_id=5 --increase_count=50 --direction=1
```

10.JogCartesian 笛卡尔点动（位置+姿态）

➢ 指令用法	笛卡尔空间连续运动

➢ 指令参数	数据类型	含义
--increase_count	Number	加速过程周期数
--moving_type	Number	笛卡尔运动方式 0~5（X、Y、Z 平动、旋转）
-- speed --speed_value/	speed	六维笛卡尔速度：x/y/z 方向平动，单位 m/s， rx/ry/rz 方向转动，单位 rad/s。 （详情见 speed 速度参数）
--direction	Number	运动方向：1-正方向运动，-1-负方向运动
--coordinate	Number	运动的参考坐标系： 0-绝对世界坐标系，1-工具坐标系

➢ **指令说明**

➢ **指令示例**

例 1

```
JogCartesian --moving_type=0 --direction=1 --speed=v100 --coordinate=0 --increase_count=50
JogCartesian --moving_type=1 --direction=-1 --speed_value=0.1 --coordinate=0 --increase_count=50
JogCartesian --moving_type=2 --direction=1 --speed=v100 --coordinate=0 --increase_count=50
```

例 2

```
JogCartesian --moving_type=3 --direction=1 --speed=v100 --coordinate=0 --increase_count=50
JogCartesian --moving_type=4 --direction=1 --speed_value=0.1 --coordinate=0 --increase_count=50
JogCartesian --moving_type=5 --direction=1 --speed=v100 --coordinate=0 --increase_count=50
```

11.JogC 单步笛卡尔点动

➢ 指令用法	笛卡尔空间单步运动

➢ 指令参数	数据类型	含义
--speed/ --speed_value	speed	六维笛卡尔速度：x/y/z 方向平动，单位 m/s，rx/ry/rz 方向转动，单位 rad/s。
--motion_type	Number	笛卡尔运动方式 0~5（X、Y、Z 平动、旋转）
--direction	Number	运动方向，1 表示正方向，-1 表示负方向
--step	Number	点动步长，指单次点动操作中，执行机构的最小移动量，x/y/z 方向平动，单位 m ，rx/ry/rz 方向转动，单位 rad (必须为正数)
--coordinate	Number	运动的参考坐标系：0-绝对世界坐标系，1-工具坐标系

➢ **指令说明**

➢ **指令示例**

例 1

JogC --motion_type=0 --speed_value=0.02 --direction=1 --coordinate=0

JogC --motion_type=1 --speed=v100 --direction=1 --coordinate=0 --step=0.1

JogC --motion_type=2 --speed_value=0.02 --direction=1 --coordinate=0

例 2

JogC --motion_type=3 --speed_value=0.02 --direction=1 --coordinate=0 --step=0.1

JogC --motion_type=4 --speed=v50 --direction=1 --coordinate=0

JogC --motion_type=5 --speed_value=0.02 --direction=1 --coordinate=0

三、在线规划指令

1.SpeedJ 在线关节速度控制

➢ 指令用法	关节空间下，SpeedJ 用于执行关节速度控制指令。用户可输入期望的关节速度，控制器根据设定的最大加速度（acc）、减速度（dec）和加加速度（jerk）进行速度插值规划，使关节速度平滑变化。 该指令属于在线速度控制指令，可周期性发送新的速度指令以实时更新关节运动状态。 当在持续时间内未收到新的 SpeedJ 指令时，系统会自动减速停止，使机械臂保持静止。

➢ 指令参数	数据类型	含义
--vel	Number	目标关节速度
--acc	Number	关节最大加速度
--dec	Number	关节最大减速度
--jerk	Number	关节最大加加速度
--last_count	Number	持续时间
--stop	无	停止当前速度控制

➢ 指令说明	1.vel 参数：目标关节速度，支持多关节数组输入，例如{v1，v2，v3，v4，v5，v6}表示各个关节目标速度；{v}表示各个关节目标速度均为 v。 2. last_count 参数：持续时间。 该指令的持续时间，当然要是在持续时间内发送下一条该指令，则直接执行下一条指令，因为这是实时更新的 3. stop 参数：SpeedJ --stop 表明立即退出在线规划状态并减速停止运动。
➢ 指令示例	例 1：在 last_count 内均可以实时更新关节速度，当最后一个指令发出后在 last_count 内没有接受新 SpeedJ 指令则退出在线规划状态减速停止运动 例 2：就算在 last_count 内，发出该 SpeedJ --stop 指令后会立刻退出在线规划状态并停止运动

例 1

```
SpeedJ --vel={0.03,0,0,0,0,0,0} --acc={1} --dec={1} --jerk={1} --last_count=1000
SpeedJ --vel={0.4,0,0,0,0,0,0} --acc={1} --dec={1} --jerk={1} --last_count=1000
SpeedJ --vel={0.1,0,0,0,0,0,0} --acc={1} --dec={1} --jerk={1} --last_count=1000
```

例 2

```
SpeedJ --vel={0.2}；//表明每个关节速度均为 0.2
SpeedJ --stop；
```

2.SpeedL 笛卡尔空间在线速度控制

➢ 指令用法	笛卡尔空间下，SpeedL 用于执行工具中心点（TCP）的线速度和角速度控制指令。用户可输入期望的笛卡尔速度向量，控制器根据设定的最大加速度（acc）和加加速度（jerk）进行速度插值规划，使 TCP 运动平滑变化。速度向量的参考坐标系由 coordinate 参数指定。 该指令属于在线速度控制指令，可周期性发送新的速度指令以实时更新 TCP 的运动状态。 当在持续时间内未收到新的 SpeedL 指令时，持续时间结束后系统会自动减速停止，使机械臂保持静止。

➢ 指令参数	数据类型	含义
--last_count	Number	指令持续时间
--vel	Number	旋量速度
--acc	Number	旋量加速度
--jerk	Number	旋量加加速度
--coordinate	Number	选择基准坐标系为世界坐标系或工具坐标系
--stop	无	停止当前速度控制

➢ 指令说明	1.vel 参数：目标笛卡尔速度，支持多关节数组输入，例如{v1，v2，v3，v4，v5，v6}表示 xyz 线速度以及绕 xyz 的角速度；{v}表示各个方向速度均为 v。 2. last_count 参数：持续时间。 该指令的持续时间，当然要是在持续时间内发送下一条该指令，则直接执行下一条指令，因为这是实时更新的。 3. stop 参数：SpeedL --stop 表明立即退出在线规划状态并减速停止运动。
➢ 指令示例	例 1：在 last_count 内均可以实时更新速度，当最后一个指令发出后在 last_count 内没有接受新 SpeedL 指令则退出在线规划状态减速停止运动 例 2：就算在 last_count 内，发出该 SpeedL --stop 指令后会立刻退出在线规划状态并停止运动

例 1

```
SpeedL --vel={0.005,0,0,0,0,0} --acc={2,2,2,2,2,2} --jerk={5,5,5,5,5,5} --last_count=5000 --coordinate=0
SpeedL --vel={-0.1,0,0,0,0,0} --acc={2,2,2,2,2,2} --jerk={5,5,5,5,5,5} --last_count=100 --coordinate=0
```

例 2

```
SpeedL --vel={0,0.1,0,0,0,0} --acc={2,2,2,2,2,2} --jerk={5,5,5,5,5,5} --last_count=100 --coordinate=1
SpeedL --stop
```

3.JogAnyJ 在线关节角度规划

➢ 指令用法	关节空间下，JogAnyJ 用于执行在线关节角度规划指令，支持用户实时输入任意关节角度目标点。控制器根据机器人允许的最大速度和最大加速度进行轨迹插补，使机械臂连续运动。

➢ 指令参数	数据类型	含义
--jointtarget/ --jointtarget_var/ --jointtarget_value	jointtarget	目标关节角度（详情见 jointtarget 关节空间参数）
--joint_vel	Number	关节最大速度系数
--joint_acc	Number	关节最大加速度系数
--joint_dec	Number	关节最大减速度系数
--last_count	Number	等待新指令周期数，单位 count，1count=0.002s
--zone_ratio	Number	轨迹平滑比例
--clear_buffer	Number	是否清除缓存队列

➢ 指令说明	1. last_count 参数：等待新指令周期数。 在指定周期内：没有新指令，退出在线规划状态，若收到新指令，继续规划运动。 2. zone_ratio 参数：控制轨迹平滑过渡。 当目标点剩余轨迹比例小于 zone_ratio：接入下一个目标点，实现平滑运动。 3. --jointtarget 使用的是示教点位的点输入相应 name 即可，默认 P0 -- jointtarget _var 使用的是 var 定义的点输入相应 name 即可，默认 P0 -- jointtarget _value 直接输入相应数值 3. clear_buffer：是否清除缓存队列。 若不启用，则依次运行之前设定的目标点位，若启用，则清除尚未执行的目标点位，并插入新的目标点位，且打断正在执行的目标点直接执行新的目标点。
➢ 指令示例	依次下发目标 ABC 例 1：机械臂运动轨迹顺序为：A → B → C 例 2：若下发目标 C 时已经开始执行目标 A 则立刻直接执行目标 C（打断了正在执行的目标 A）

例 1

```
JogAnyJ --jointtarget_value={0,-1,-1.57,-1.57,1.57,0,0,0,0,0} --clear_buffer=0
JogAnyJ -- jointtarget_value ={0,-1,-1.0,-1.57,1.57,0,0,0,0,0} --clear_buffer=0
JogAnyJ -- jointtarget_value ={0,-1,-1.57,-1.0,1.57,0,0,0,0,0} --clear_buffer=0
```

例 2

```
JogAnyJ --jointtarget_value={0,-1,-1.57,-1.57,1.57,0,0,0,0,0} --clear_buffer=0
JogAnyJ -- jointtarget_value ={0,-1,-1.0,-1.57,1.57,0,0,0,0,0} --clear_buffer=0
JogAnyJ -- jointtarget_value ={0,-1,-1.57,-1.0,1.57,0,0,0,0,0} --clear_buffer=1
```

4.JogAnyC 在线笛卡尔位置规划

➢ 指令用法	笛卡尔空间下，JogAnyC 用于执行在线末端位置规划指令，支持实时输入三维位置目标点。控制器根据机器人允许的最大速度和加速度进行轨迹插补，使机械臂连续运动。

➢ 指令参数	数据类型	含义
--robottarget/ --robottarget_var/ robottarget_value	robottarget	目标点位姿信息
--cartesian_vel	Number	末端最大速度系数
--cartesian_acc	Number	末端最大加速度系数
--cartesian_dec	Number	末端最大减速度系数
--last_count	Number	等待新指令周期数，单位 count，1count=0.002s
--zone_ratio	Number	轨迹平滑比例
--clear_buffer	Number	是否清空缓存队列

➢ 指令说明	1 执行该指令时姿态也会随目标点变化 2. –robottarget 使用的是示教点位的点输入相应 name 即可，默认 P0 --robottarget_var 使用的是 var 定义的点输入相应 name 即可，默认 P0 --robottarget_value 直接输入相应数值 2. last_count 参数：等待新指令周期数。 在指定周期内：没有新指令，退出在线规划状态，若收到新指令，继续规划运动。 3. zone_ratio 参数：控制轨迹平滑过渡。 当目标点剩余轨迹比例小于 zone_ratio：接入下一个目标点，实现平滑运动。 4. clear_buffer：是否清空缓存队列。 若不启用，则依次运行之前设定的目标点位，若启用，则清除尚未执行的目标点位，并插入新的目标点位，且打断正在执行的目标点直接执行新的目标点。
➢ 指令示例	依次下发目标 ABC 例 1：机械臂运动轨迹顺序为：A → B → C 例 2：若下发目标 C 时已经开始执行目标 A 则立刻直接执行目标 C（打断了正在执行的目标 A）

例 1

```
JogAnyC --robottarget_value={0.6,-0.25,0.3,-0.5, 0.5, -0.5, 0.5} --zone_ratio=0.05 --clear_buffer=0
JogAnyC --robottarget_value={0.6,-0.25,0.64,0.62721137,-0.32650558,0.32650558,-0.62721137}
--zone_ratio=0.05 --clear_buffer=0
JogAnyC --robottarget_value={0.6,0.1,0.64,-0.5, 0.5, -0.5, 0.5} --zone_ratio=0.05 --clear_buffer=0
```

例 2

```
JogAnyC --robottarget_value={0.6,-0.25,0.3,-0.5, 0.5, -0.5, 0.5} --zone_ratio=0.05 --clear_buffer=0
JogAnyC --robottarget_value={0.6,-0.25,0.64,0.62721137,-0.32650558,0.32650558,-0.62721137}
--zone_ratio=0.05 --clear_buffer=0
JogAnyC --robottarget_value={0.6,0.1,0.64,-0.5, 0.5, -0.5, 0.5} --zone_ratio=0.05 --clear_buffer=1
```

5.JogAnyCpos 在线笛卡尔位置规划

➢ **指令用法**	笛卡尔空间下，JogAnyCpos 用于执行在线末端位置控制指令，支持基于当前位置的实时点动控制。控制器根据设定的速度、加速度对当前 TCP 位置进行直接驱动，使机械臂按指令连续运动。

➢ **指令参数**	数据类型	含义
--cartesian_pos	Number	目标点位置信息
--cartesian_vel	Number	末端最大速度系数
--cartesian_acc	Number	末端最大加速度系数
--cartesian_dec	Number	末端最大减速度系数
--last_count	Number	等待新指令周期数，单位 count，1count=0.002s
--zone_ratio	Number	轨迹平滑比例
--clear_buffer	Number	是否清空缓存队列

➢ **指令说明**	1. --cartesian_pos 参数是位置信息{x,y,z}，单位是 m 2. last_count 参数：等待新指令周期数。 在指定周期内：没有新指令，退出在线规划状态，若收到新指令，继续规划运动。 3. zone_ratio 参数：控制轨迹平滑过渡。 当目标点剩余轨迹比例小于 zone_ratio：接入下一个目标点，实现平滑运动。 4. clear_buffer：是否清空缓存队列。 若不启用，则依次运行之前设定的目标点位，若启用，则清除尚未执行的目标点 位，并插入新的目标点位，且打断正在执行的目标点直接执行新的目标点。
➢ **指令示例**	依次下发目标 ABC 例 1：机械臂运动轨迹顺序为：A → B → C 例 2：若下发目标 C 时已经开始执行目标 A 则立刻直接执行目标 C（打断了正在执行的目标 A）

例 1

```
JogAnyCpos --cartesian_pos={0.6,-0.25,0.3}
JogAnyCpos --cartesian_pos={0.6,-0.25,0.6} --zone_ratio=0.05 --clear_buffer=0
JogAnyCpos --cartesian_pos={0.42,-0.28,0.74} --clear_buffer=0
```

例 2

```
JogAnyCpos --cartesian_pos={0.6,-0.25,0.3,-0.5, 0.5, -0.5, 0.5} --zone_ratio=0.05 --clear_buffer=0
JogAnyCpos --cartesian_pos={0.6,-0.25,0.64, } --zone_ratio=0.05 --clear_buffer=0
JogAnyCpos --cartesian_pos={0.6,0.1,0.64,-0.5, 0.5, -0.5, 0.5} --zone_ratio=0.05 --clear_buffer=1
```

6.SetEgmOffset 在线偏移控制

➢ 指令用法	用于设置 EGM 在线偏移量，在机器人运动过程中对规划轨迹进行在线修正

➢ 指令参数	数据类型	含义
--pos_offset	Matrix	位置偏移量，用于设置机器人末端的目标偏移量
--vel_offset	Matrix	速度偏移量，用于设置机器人末端的速度偏移量
--egmplanner_id	Number	EGM 规划器编号，用于指定当前使用的 EGM planner
--max_vel	Matrix	最大速度限制
--max_acc	Matrix	最大加速度限制
max_dec	Matrix	最大减速度限制
coordinate	Number	偏移参考坐标系，0 表示基坐标系，1 表示工具坐标系
control_mode	Number	控制模式选择，用于指定当前在线偏移的控制方式

➢ 指令说明	SetEgmOffset 用于在线修改机器人运动过程中的偏移量，可用于轨迹微调、实时补偿等场景 pos_offset 表示位置偏移量，vel_offset 表示速度偏移量，二者可根据实际控制需求进行设置。 max_vel、max_acc、max_dec 用于限制偏移过程中的速度、加速度和减速度，避免偏移过程过快导致运动不平滑
➢ 指令示例	例 1：设置机器人在基坐标系下对 x 偏移 0.01m 例 2：设置机器人在工具坐标系下进行速度偏移

例 1

```
SetEgmOffset --pos_offset={0.01,0,0} --max_vel=0.005 --max_acc=0.3 --max_dec=0.3 --coordinate=0
```

例 2

```
SetEgmOffset --vel_offset={0.003,0,0} --max_vel=0.005 --max_acc=0.3 --max_dec=0.3 --coordinate=1
```

四、常用系统指令

1.Clear 错误清理

➢ 指令用法	清除异常等级为 ERROR 的异常，恢复系统到正常状态
➢ 指令参数	无
➢ 指令说明	用于报错等级为 ERROR 时，清除错误信息，并不会清除变量 Var；
➢ 指令示例	无

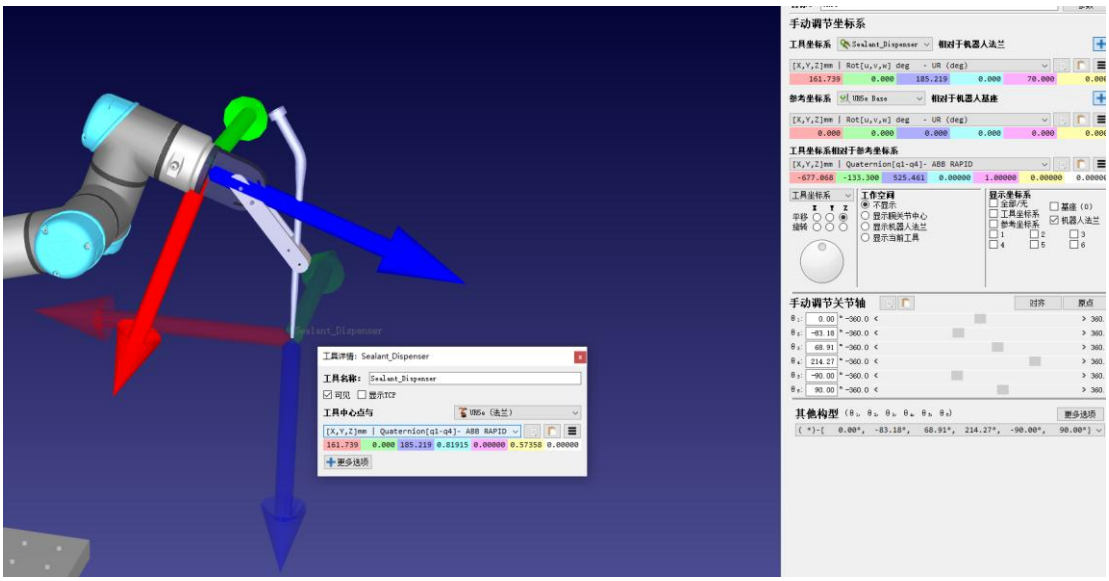
2.SetTool 设置工具

➢ 指令用法	设置末端工具坐标系	
➢ 指令参数	数据类型	含义
--tool_var	tool	工具坐标系位姿信息
➢ 指令说明		
➢ 指令示例		

例 1

```
Var --type=tool --name=tool_new --value={0.161739,0,0.185219,0,0.57358,0,0.81915}
SetTool --tool_var=tool_new
```

注意：在末端法兰盘坐标系下，期望的新 tool 的坐标


手动调节坐标系
工具坐标系
Sealant_Dispenser
相对于机器人法兰
[X,Y,Z]mm | Rot[u,v,w] deg - UR (deg)
161.739 0.000 185.219 0.000 70.000
参考坐标系
UR5e Base
相对于机器人基座
0.000 0.000 0.000 0.000 0.000
工具坐标系相对于参考坐标系
[X,Y,Z]mm | Quaternion[q1-q4]- ABB RAPID
-677.068 -133.300 525.461 0.00000 1.00000 0.00000
工具坐标系
平移
旋转
工作空间
不显示
显示耦关节中心
显示机器人法兰
显示当前工具
显示坐标系
全部/无
基座 (0)
工具坐标系
机器人法兰
参考坐标系
手动调节关节轴
对齐
原点
其他构型
更多选项
工具详情: Sealant_Dispenser
工具名称:
Sealant_Dispenser
可见
显示TCP
工具中心点与
UR5e (法兰)
[X,Y,Z]mm | Quaternion[q1-q4]- ABB RAPID
161.739 0.000 185.219 0.81915 0.00000 0.57358 0.00000
更多选项

3.SetWobj 设置基坐标系

➢ 指令用法	设置基坐标系

➢ 指令参数	数据类型	含义
-- wobj_var	wobj	基坐标系位姿信息

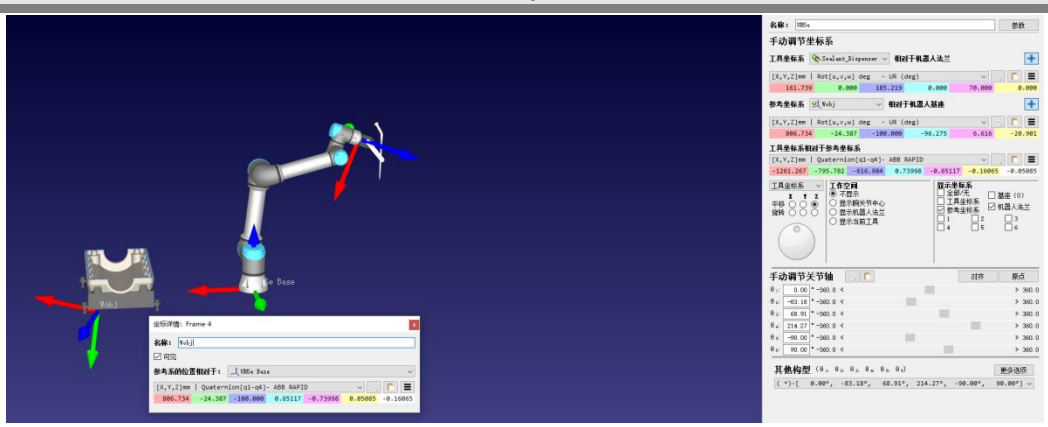
➢ 指令说明

➢ 指令示例

例 1

```
Var --type=wobj --name=wobj_new --value={0.392367,-0.024387,0,0,0,0,0}
SetWobj --wobj_var=wobj_new
```

注意：在 base 坐标系下，期望的新 wobj 的坐标



4.SetUsingSP 最优规划求解

➢ 指令用法	最优规划求解模式

➢ 指令参数	数据类型	含义
--state		开关

➢ 指令说明

➢ 指令示例

例 1

```
SetUsingSP --state=on;//打开最优求解器
SetUsingSP --state=off;//关闭最优求解器
```

5.SubLoop 多模型控制优化指令

➢ 指令用法	多模型控制优化

➢ 指令参数	数据类型	含义
--exec	String	当前模型需要执行的指令
--immediate	String	指令立即执行

➢ 指令说明

该指令在多模型场景下，对系统控制指令的处理机制进行了优化，具体如下：

在多模型场景中，如果不使用 SubLoop，而是直接发送组合指令，例如：{MoveAbsJ||MoveAbsJ}，那么系统只有在整条大指令全部执行完成并返回结果之后，才能继续发送下一条指令。也就是说，在这种方式下，必须等两个模型对应的小指令都执行结束，系统才会进入下一步。

当两个小指令的执行耗时接近时，这种方式问题不大；但如果两条小指令的执行时间差异较大，系统就会被执行较慢的那条指令拖住，导致整体返回时间变长，影响后续指令的下发效率，这显然不够合理。

因此，引入了 SubLoop 机制来解决这一问题。

在使用 SubLoop 后，如{SubLoop --exec={MoveAbsJ}||SubLoop --exec={MoveAbsJ}}，只要后端收到了 SubLoop 指令，会将大指令中的小指令放到对应模型的待执行队列中，然后立刻响应一个返回值（第一条指令除外）。后端会有对应的线程去处理待执行队列中的指令。与此同时，后续到来的新指令不会丢失，而是会先进入待执行队列；等当前对应的小指令执行完成后，再从队列中依次取出并继续执行。

这样一来，系统在多模型场景下的指令处理会更加高效，也能避免因个别慢指令而阻塞整体流程。

如图所示：

```
Input commands:
{SubLoop --exec={Recover}||SubLoop --exec={Recover}}
>>>1 [async] msg: {SubLoop --exec={Recover}||SubLoop --exec={Recover}}
Input commands:
{SubLoop --exec={MoveAbsJ}||SubLoop --exec={MoveAbsJ}}
>>>1 [async] msg: {SubLoop --exec={MoveAbsJ}||SubLoop --exec={MoveAbsJ
**************Async**************
model size:2
subcmd_index:0
return_code:0
return_message:
subcmd_index:1
return_code:0
return_message:
********************************
Input commands:
{SubLoop --exec={exit}||SubLoop --exec={exit}}
>>>1 [async] msg: {SubLoop --exec={exit}||SubLoop --exec={exit}}
**************Async**************
model size:2
subcmd_index:0
return_code:0
return_message:
subcmd_index:1
return_code:0
return_message:
********************************
**************Async**************
model size:2
subcmd_index:0
return_code:1
return_message:
subcmd_index:1
return_code:1
return_message:
********************************
```

➢ 注意事项

1）在使用 SubLoop 时，通常发送的第一条 SubLoop 指令是不会立刻有返回值的，后续指令都会有返回值，第一条指令的返回值是在 SubLoop 退出的时候，这时会返回 SubLoop 的第一条指令的返回值。

2）第一条指令必须是异步指令且超时时间必须长于接下来执行所有指令时间的总和，后续的指令可以是同步也可以是异步。

3）使用 SubLoop 时，若处在多模型中只能发送多模型指令，不可对其中一个模型发指

令，另一个不发。

4）默认情况下，immediate 为 false

➢ 指令示例

```
# 双模型指令，对第一个模型发送 Recover 指令，第二个模型不发送指令（错误示例）
SubLoop --exec={Recover}||NotRunExecute

# 双模型指令，对两个模型发送指令（正确示例）
SubLoop --exec={Move1}||SubLoop --exec={Move2}

# immediate 置为 true，则系统会放弃对当前指令的处理，处理这个指令
SubLoop --exec={getInfo} --immediate=true
```

五、高级规划指令

可实现直线、圆弧轨迹。七轴情况下需要开启“最优求解”模式, 参考“系统指令”：SetUsingSP。

1.MoveBlend 笛卡尔空间下轨迹融合(局部)

➢ **指令用法**	笛卡尔空间下，MoveBlend 用于对相邻两段运动之间的拐点进行局部轨迹融合，通过在拐角处引入过渡曲线（如圆弧或平滑段），避免速度突变和频繁启停。

➢ **指令参数**	数据类型	含义
--robottarget/ --robottarget_var/ --robottarget_value	robottarget	目标点位置变量(详情见 robottarget 笛卡尔空间参数)
--mid_robottarget/ --mid_robottarget_var/ --mid_robottarget_value/	robottarget	中间点位置变量（调用和 robottarget 笛卡尔空间参数相同）
--speed/ --speed_var/ --speed_value/	speed	速度变量（详情见 speed 速度参数）
--type		插值类型，默认 insert
--zone	zone	轨迹融合变量(详情见 zone 转弯半径)
--ratio	Number	控制拐点前后轨迹参与融合的相对比例与平滑程度
--is_optimal	Number	当使用 MoveBlendOptimalpos 时，参数设为 1

➢ **指令说明**	1.zone 参数：第一个参数（单位 m）表示空间过渡容差，越小越接近原来轨迹，允许轨迹在空间上“偏离目标点”的最大距离 2.轨迹点姿态可以变化，执行轨迹时姿态变化也会执行，但有姿态变化时，速度会降低（比较慢） 3.运动类型，type=first_insert 当前位置为起点标记类型（固定的）后面不用加其他参数，type=insert_line 直线轨迹点类型，type=insert_circle 圆弧轨迹点类型，type=start 执行的类型（固定的）后面不用加其他参数 4.每段速度可以单独决定，由标记字段指令 speed 决定
➢ **指令示例**	**例 1** 笛卡尔空间下轨迹融合直线运动 **例 2** 笛卡尔空间下轨迹融合圆弧运动 **例 3** 笛卡尔空间下轨迹融合圆弧运动(整圆)

例 1

```
Var --type=robottarget --name=P1 --value={0.036, 0, 0.9,0, 0, 0, 1};//定义轨迹点位姿
Var --type=robottarget --name=P2 --value={0.036, -0.068, 0.9, 0, 0, 0, 1 };
Var --type=robottarget --name=P3 --value={-0.083, -0.068, 0.9,0,0, 0, 1};
MoveBlend --type=first_insert  //起点
MoveBlend --type=insert_line --robottarget_var=P1 --zone={0,0} --speed=v100//直线轨迹点标
```

记起点到 P1

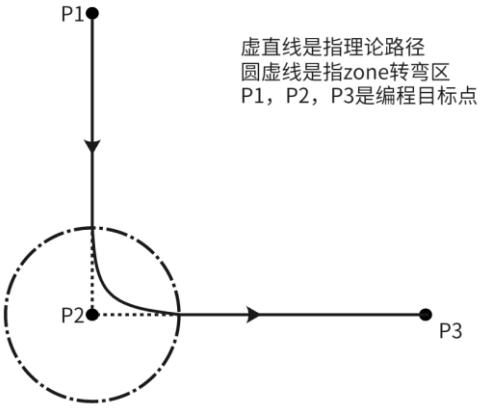
MoveBlend --type=insert_line --robottarget_var=P2 --zone={0.05,0.25} --speed=v100//zone 越大，可脱离原轨迹程度越大，zone 转弯区直径为 0.05m

MoveBlend --type=insert_line --robottarget_var=P3 --zone={0,0} --speed=v100

MoveBlend --type=start //执行命令，速度不由这一行决定，由标志点决定

//其他省略的参数均为默认值

笛卡尔空间下轨迹融合直线运动如图下所示



例 2

Var --type=robottarget --name=p1 --value={0.32, -0.32, 0.48,0, 1, 0, 0};//定义轨迹点位姿

Var --type=robottarget --name=p2 --value={0.36, -0.28, 0.48,0, 1, 0, 0};

Var --type=robottarget --name=p3 --value={0.40, -0.32, 0.48,0, 1, 0, 0};

MoveBlend --type=first_insert //起点

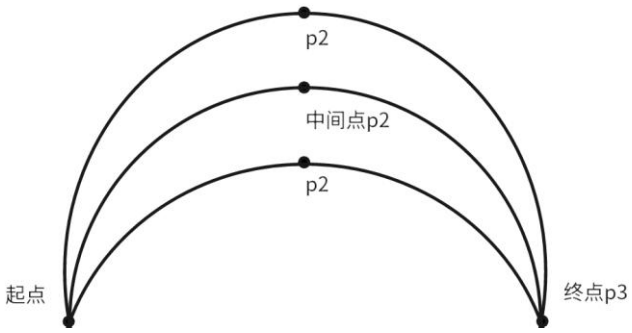
MoveBlend --type=insert_circle --mid_robottarget_var=p2 --robottarget_var=p3 --zone={0.1,0.1} --speed=v300

//圆弧轨迹点标记，zone 越大，可脱离原轨迹程度越大，以 p2 作为中间点，圆弧运动到 p3

MoveBlend --type=start //执行命令，速度不由这一行决定，由标志点决定

//其他省略的参数均为默认值

笛卡尔空间下轨迹融合圆弧运动如图下所示



例 3 笛卡尔空间下轨迹融合圆弧运动(整圆)

Var --type=robottarget --name=z1 --value={0.524, -0.15,0.684,-0.5,0.501,-0.499,0.498};//定义轨迹点位姿

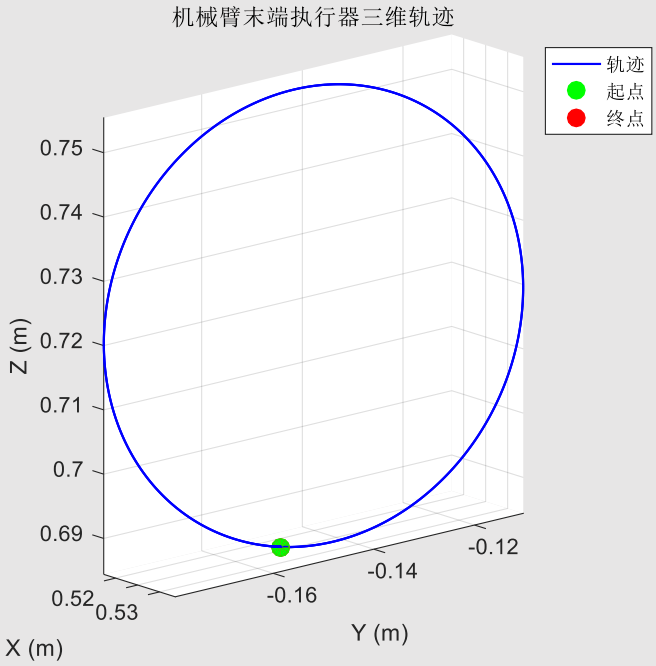
Var --type=robottarget --name=z2 --value={0.523, -0.15,0.754,-0.483,0.489,-0.516,0.509};

MoveBlend --type=first_insert //需要机械臂末端位姿在运动前就在 z1 点

MoveBlend --type=insert_circle --robottarget_var=z1 --mid_robottarget_var=z2 --speed=v30 --zone={0,0} --is_optimal=0 //z1 点为终点，z2 点为中间点，为末端轨迹是确保完整的圆，zone 值建议为 0

MoveBlend --type=start　//执行命令

笛卡尔空间下轨迹融合圆弧运动(整圆)如图下所示

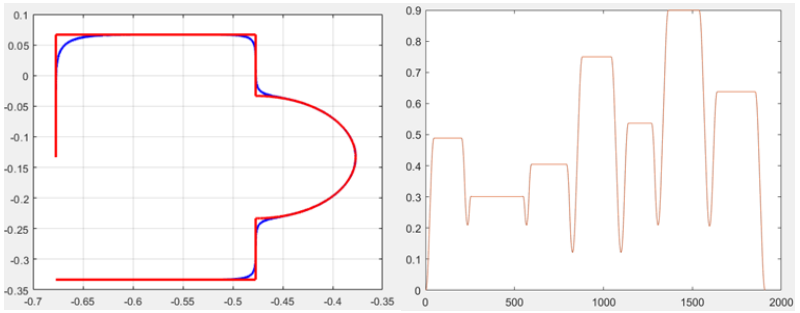


➢ **轨迹特性说明对比**

下图展示了一条由直线与圆弧组合而成的基础路径。在未启用 zone 轨迹融合时（类似于采用 MoveL 指令），相邻路径段在连接处存在明显拐角，机械臂需要在该位置进行减速甚至停顿，轨迹在运动学上不满足平滑连续（红色曲线）。

当启用 zone 轨迹融合机制后，控制器会在路径连接处自动插入过渡曲线，使相邻路径段实现平滑过渡，从而形成连续光滑的运动轨迹（蓝色曲线）。

同时，从速度曲线可以看出，在启用 zone 融合 后，机械臂在路径连接处不会降低至 0 速度 再启动下一段轨迹，而是在满足约束条件的情况下保持连续速度过渡，从而减少频繁启停，提高整体运动效率与轨迹平滑性。

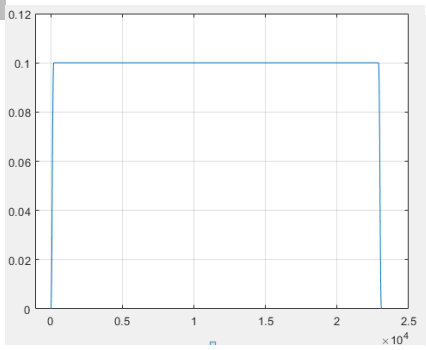
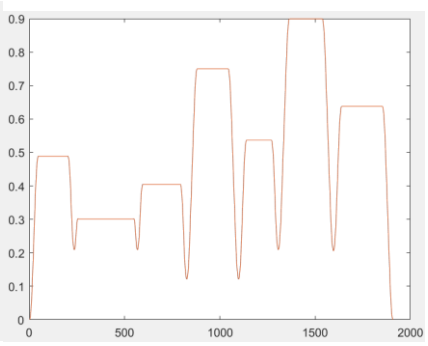

0.1
0.05
0
-0.05
-0.1
-0.15
-0.2
-0.25
-0.3
-0.35
-0.7
-0.65
-0.6
-0.55
-0.5
-0.45
-0.4
-0.35
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
0
500
1000
1500
2000

2.MoveBlendScurve 笛卡尔空间下轨迹融合（匀速）

➢ 指令用法	笛卡尔空间下，机械臂末端执行基于 S 曲线插值算法的融合轨迹运动，用于实现复杂运动中，运动线速度按照制定曲线运动，适用于涂胶、打磨等恒速情形下。

➢ 指令参数	数据类型	含义
--speed/--speed_var/--speed_value	speed	速度变量（详情见 speed 速度参数）
--acc	Number	笛卡尔加速度，单位为 rad/s/s
--jerk	Number	笛卡尔加加速度，单位为 rad/s/s/s

➢ 指令说明	1.zone 参数：第一个参数（mm）表示空间过渡容差，越小越接近原来轨迹，允许轨迹在空间上“偏离目标点”的最大距离 第二个参数表示姿态过渡容差，允许姿态不完全对齐就进入下一段轨迹 2.轨迹仅执行位置变化，就算轨迹点姿态变化了，也不会执行姿态变化，但也不会报错 3.运动类型，type=first_insert 当前位置为起点标记类型（固定的）后面不用加其他参数，type=insert_line 直线轨迹点类型，type=insert_circle 圆弧轨迹点类型，MoveBlendScurve 执行 4.每段速度不可以单独决定，由 MoveBlendScurve 指令 speed 决定

➢ 规划特性说明对比	左图为 MoveBlendScurve 执行时的末端线速度变化曲线。可以看出，末端线速度按照设定的 S 曲线规律进行加速与减速，速度变化过程连续且平滑。 相比 MoveBlend 的速度规划方式（右图），MoveBlendScurve 对末端线速度进行了显式规划，使速度曲线呈现规则的 S 型变化，避免速度突变或波动，从而获得更加稳定的进给速度。因此，该指令更适用于对末端线速度稳定性要求较高的工艺场景，例如涂胶、打磨、喷涂等。  

➢ 跑固定曲线说明	该指令为分段插值方式，连续曲线必须先离散为多个点。步骤如下 1. 曲线离散化（最好时间参数化），将连续曲线转换为一组有序且连续的 robottarget 点 2. 使用 --type=first_insert 标记当前位姿为起点 3. 使用 --type=insert_line 依次插入离散点。 4. 使用 MoveBlendScurve 指令统一设定速度并执行轨迹 注意事项：1.点与点空间距离不小于 2mm 2.单次轨迹点建议不要超过 300 个 3.zone 尽可能设小点比如{0.001,0.01}，越小越接近原轨迹 4.建议起点为第一个点，否则可能会报二次不连续

➢ **指令示例**

例 1

```
Var --type=robottarget --name=p1 --value={0.491, -0.1, 0.687,-0.5, 0.5, -0.5, 0.5};//定义轨迹点位姿
Var --type=robottarget --name=p2 --value={0.491, -0.0, 0.687,-0.5, 0.5, -0.5, 0.5};
Var --type=robottarget --name=p3 --value={0.491, 0.02, 0.587，-0.5, 0.5, -0.5, 0.5};
MoveBlend --type=first_insert  //起点
MoveBlend --type=insert_line --robottarget_var=p1 --zone={0.1,0.1}//直线轨迹点标记
MoveBlend --type=insert_line --robottarget_var=p2 --zone={0.1,0.1};
MoveBlend --type=insert_line --robottarget_var=p3 --zone={0.1,0.1};
MoveBlendScurve --speed_var=v100 //执行命令，速度仅由这一行决定
//其他省略的参数均为默认值
```

3.MoveBlendOptimalpos 笛卡尔空间下轨迹融合（最大速）

➢ 指令用法

笛卡尔空间下，机械臂末端执行融合轨迹运动，生成的轨迹在满足关节电机最大速度、最大加速度、最大力矩的前提下，尽快的完成运动。

➢ 指令参数

	数据类型	含义
--acc_coef	Number	加速度系数,值越大,轨迹加速越快
--vel_coef	Number	速度系数，值越大，运行越快
--dynamics_constraint	Number	是否启用机器人动力学约束。1 表示启用（默认），0 表示忽略约束。
--vel_gain	Number	优化中速度约束的调节增益
--acc_gain	Number	优化中加速度约束的调节增益

➢ 指令说明

1.轨迹点姿态不要变化，否则会影响到计算，可能会失败

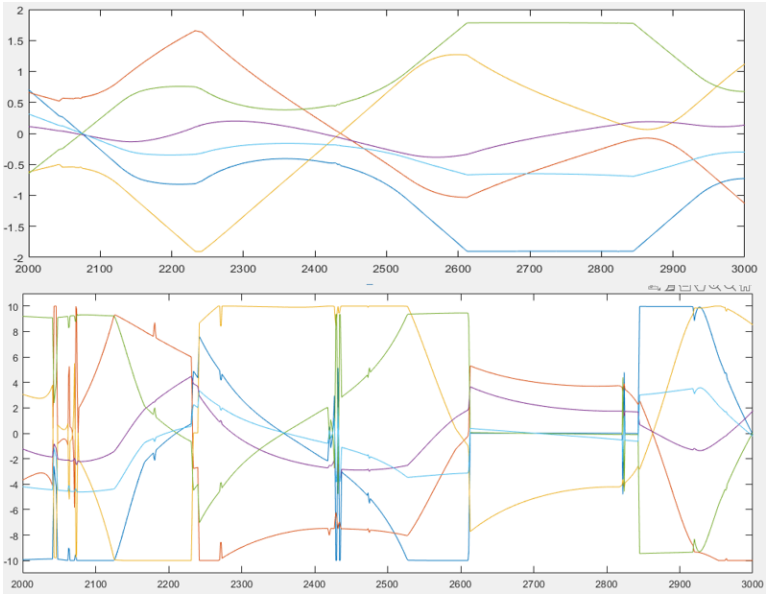
2.运动类型，type=first_insert 当前位置为起点标记类型（固定的）后面不用加其他参数，type=insert_line 直线轨迹点类型，type=insert_circle 圆弧轨迹点类型，MoveBlendScurve 执行

3.采用此指令时，标记点字段需要加上--is_optimal=1

➢ 轨迹特性说明

时间最优规划生成的轨迹在满足关节最大速度、最大加速度及最大力矩约束的前提下，尽可能缩短整体运动时间。

下图为各关节的速度曲线和加速度曲线。可以看出，在运动过程中始终存在某一关节的速度或加速度接近其约束上限，说明轨迹规划充分利用了机器人动力学能力，从而实现时间最优运动。



- **指令示例**

例 1

```
Var --type=robottarget --name=p1 --value={0.491, -0.1, 0.687,-0.5, 0.5, -0.5, 0.5};//定义轨迹点位姿
Var --type=robottarget --name=p2 --value={0.491, -0.0, 0.687,-0.5, 0.5, -0.5, 0.5};
Var --type=robottarget --name=p3 --value={0.491, 0.02, 0.587，-0.5, 0.5, -0.5, 0.5};
MoveBlend --type=first_insert //起点标记
MoveBlend --type=insert_line --robottarget_var=p1 --zone={0.1,0.1} --is_optimal=1;//直线轨迹点标记
MoveBlend --type=insert_line --robottarget_var=p2 --zone={0.1,0.1} --is_optimal=1;
MoveBlend --type=insert_line --robottarget_var=p3 --zone={0.1,0.1} --is_optimal=1;//采用示教点位时，加上这个参数
MoveBlendOptimalPos --acc_coef={0.9} --vel_coef={0.9} --dynamics_constraint=0;
//执行命令
//其他省略的参数均为默认值
```

4.MoveS 笛卡尔空间下轨迹融合（全程）

➢ 指令用法	笛卡尔空间下，用于对多个路径点进行整体样条拟合，生成一条全局连续的平滑轨迹，属于全局融合

➢ 指令参数	数据类型	含义
--robottarget/ --robottarget_var/ --robottarget_value/	robottarget	目标点位置变量(详情见 robottarget 笛卡尔空间参数)
--speed/ --speed_var/ --speed_value/	speed	速度变量（详情见 speed 速度参数）
--type		插值类型，默认 insert
--ratio	Number	全局平滑权重，决定平滑程度

➢ 指令说明

1. MoveS 生成的轨迹在路径点处通常不会严格经过所有路径点，而是在保证轨迹平滑性的前提下进行整体拟合。

2.轨迹点姿态可以变化，执行轨迹时姿态变化也会执行，但有姿态变化时，速度会降低（比较慢）

3.运动类型，type=first_insert 当前位置为起点标记类型（固定的）后面不用加其他参数，type=insert 轨迹点标记，type=start 执行

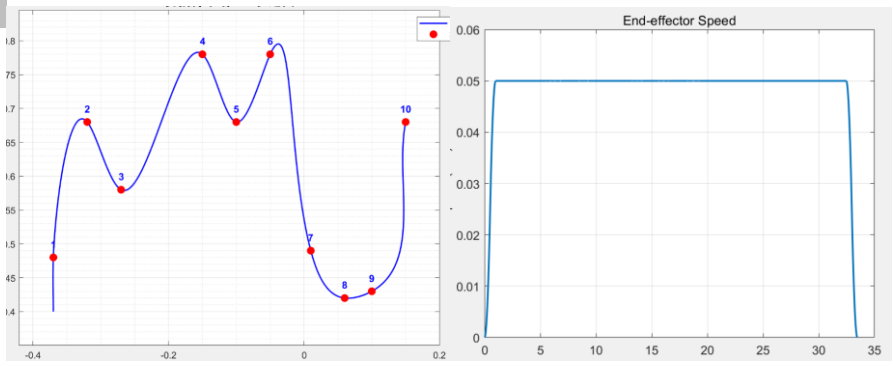
4.每段速度可以不单独决定，由 MoveS --type=start 后 speed 决定

➢ 轨迹特性说明

左图为 MoveS 执行时生成的末端运动轨迹，可以看出轨迹通过对所有路径点进行整体样条拟合后形成一条连续平滑的空间曲线。在路径点处不会形成明显拐角，而是在路径点附近进行平滑过渡，因此轨迹通常不会严格经过所有路径点。

右图为对应的末端线速度变化曲线，可以看出末端速度在整个运动过程中保持连续变化，在路径点处不会出现明显的速度突变，从而使机器人运动更加平滑稳定。

因此，MoveS 更适用于由大量离散点构成的连续曲线轨迹，例如圆弧曲线或复杂自由曲线等场景



➢ 跑固定曲线说明	该指令为全局样条拟合方式，适合连续曲线运行。步骤如下： 1. 曲线离散化（建议按时间参数化），生成一组有序 robottarget 点。 2. 使用 --type=first_insert 标记当前位姿为起点。 3. 使用 --type=insert 依次插入所有离散点。 4. 使用 --type=start 并设置 speed 与 ratio 执行轨迹并开始执行 注意事项：1.点与点空间距离不小于 2mm 2.单次轨迹点建议不要超过 320 个 3.点数较少且线比较曲折，起点最好不要是第一个点

➢ **指令示例**

例 1

```
Var --type=robottarget --name=p1 --value={0.491, -0.1, 0.687,-0.5, 0.5, -0.5, 0.5}//定义轨迹点位姿
Var --type=robottarget --name=p2 --value={0.491, -0.0, 0.687,-0.5, 0.5, -0.5, 0.5}
Var --type=robottarget --name=p3 --value={0.491,0.02, 0.587,-0.5, 0.5, -0.5, 0.5}
MoveS --type=first_insert  //起点
MoveS --type=insert --robottarget_var=p1//轨迹点标记
MoveS --type=insert --robottarget_var=p2//轨迹点标记
MoveS --type=insert --robottarget_var=p3//轨迹点标记
MoveS --type=start --speed=v200  //执行命令，速度仅由这一行决定，不由轨迹点速度决定
//其他省略的参数均为默认值
```

5.ServoSeries 关节空间下轨迹融合指令

➢ **指令用法**

关节空间连续运动，连续插值轨迹控制指令，用于向控制器发送一系列关节空间目标点，实现机械臂轨迹的平滑执行，适用于实时控制和柔顺运动场景。

➢ **指令参数**

	数据类型	含义
--joint_set	jointtarget	需要完成的期望关节位置列表
--time	Number	需要完成对应期望关节位置的时间列表
--stop_time	Number	在完成所有关节位置后留下的减速时间
--scale	Number	时间缩放比例，设置的时间列表*scale 是最终执行的事件列表

➢ **指令说明**

用户设置一系列的期望关节位置和其对应的时间列表，指令对于不同关节位置间进行插值，在设置的时间点完成对应的期望关节位置。

➢ **指令示例**

例 1 中，设置了 3 组期望关节位置，首先需要设置初始位置为起始点，然后开始 servoseries 指令，缩放比例为 1，先花费 1 秒到达第一个点位，再花费 3.2 秒到达第二个点位，再花费 6.3 秒到达第三个点位。

例 1

```
ServoSeries --type=first_insert
ServoSeries --type=insert --jointtarget_value={0,-1,-1.57,-1.57,1.57,0,0,0,0,0} --time=1
ServoSeries --type=insert --jointtarget_value={1,-1,-1.57,-1.57,1.57,0,0,0,0,0} --time=3.2
ServoSeries --type=insert --jointtarget_value={0,-1,-1.57,1.57,-1.57,0,0,0,0,0} --time=6.3
ServoSeries --type=start
```

6.MoveSeriesToppJ 关节空间下轨迹融合（最大速）

➢ 指令用法	关节空间下，用于以关节空间轨迹的形式，按照时间最优方式执行一系列关节点，确保在速度和加速度约束下，实现最快的轨迹跟踪。

➢ 指令参数	数据类型	含义
-- jointtarget --jointtarget_var/ --jointtarget_value	jointtarget	需要完成的期望关节位置列表（详情见 jointtarget 关节空间参数）
--type		插入类型
--acc_coef	Number	加速度系数，范围为(0, 1],值越大,轨迹加速越快 (默认 0.95)
--vel_coef	Number	速度系数，范围为(0, 1]，值越大，运行越快 (默认 0.95)

➢ 指令说明	
	1.轨迹点应事先设定为顺序连续的姿态/关节数据，避免大角度跳变； 2.点之间不能出现突变或不可达配置，否则插值会失败。 3.value 是关节角度信息，定义的标准格式为 10 个，数值为空时填 0。

➢ **指令示例**

例 1

```
Var --type=jointtarget --name=j1 --value={0.0,-1.57,-1.57,-1.57,1.57,0.0,0,0,0,0}
Var --type=jointtarget --name=j2 --value={0.0,-1.57,-1.13,-1.57,1.57,0.0,0,0,0,0}
Var --type=jointtarget --name=j3 --value={0.52,-1.57,-1.13,-1.57,1.57,0.0,0,0,0,0}
MoveSeriesToppJ --type=first_insert
MoveSeriesToppJ --type=insert –jointtarget_var=j1
MoveSeriesToppJ --type=insert --jointtarget_var =j2
MoveSeriesToppJ --type=insert --jointtarget_var =j3
MoveSeriesToppJ --type=start
```

六、力控指令

1.六维力传感器概述

功能介绍

六维力/力矩传感器用于实时测量机械臂末端的三维力（Fx,Fy,Fz）和三维力矩（Mx,My,Mz），支持力控与柔顺操作，主要功能包括：

- 实时采集力/力矩数据，用于柔顺控制和力控运动。
- 支撑 DragFtInCartesian 等指令。
- 提供零漂补偿与标定，保证静止状态下输出接近零。
- 可设置死区与力阈值，提高操作安全性。

安装指南

- 安装在机械臂末端执行器与工具之间，保证传感器与工具坐标系重合且测量轴线要对齐。
- 固定牢靠，避免松动或额外扭矩影响测量精度。
- 电缆连接稳固，避免拉伸或干扰。
- 设置工具坐标系 SetTool 以匹配实际工具方向。

标定与零漂

- 标定目的：消除初始偏置，保证静止输出接近零。
- 标定方法：执行 CalibFtDyn 指令，机械臂缓慢运动采集数据计算零点。
- 零漂补偿：标定后控制系统自动减去偏置值。

注意事项：机械臂静止、无外力干扰；更换工具或平台需重新标定。

2.ForcePositionHybridControl 笛卡尔空间下力-位混合控制

➢ **指令用法**

笛卡尔空间下，机器人末端沿设定方向感受到外力时做出运动响应，形成柔顺拖动控制（基于六维力的拖动示教），开启位置控制后，可以让机器人在移动过程中具备柔顺性，对外力进行反馈。

➢ **指令参数**

	数据类型	含义
--force_direction	Matrix	允许进行拖动的自由度（方向），1 表示启用，0 表示禁用
--damping_gain	Matrix	各个自由度（方向）拖动示教灵敏度参数，值越大越易拖动
--stiffness_gain	Matrix	各个自由度（方向）的刚度参数，用于描述末端在受力偏移后回到参考位置的趋势，值越大，回正能力越强，柔顺性相对减弱
--force_target	Matrix	力控自由度的期望力
--vel_limit	Matrix	拖动过程中的速度限制
--acc_limit	Matrix	拖动过程中的加速度限制
--open_tg	Matrix	1 表示启用力-位置混合控制，0 表示禁用
--coordinate	Number	拖动时参考坐标系，1 为工具坐标系（默认），0 为基坐标系
--cutoff_freq	Number	六维力传感器数据一阶低通滤波器

➢ **指令说明**

要将六维力传感器坐标系与工具坐标系对齐，否则方向可能会不对；该指令可以支持运行中，再发送新的 ForcePositionHybridControl 指令

➢ **指令示例**

例 1 中，开启 6 个方向的拖动，期望力为 0，就可以对机械臂进行笛卡尔空间下拖动。

例 2 中，开启 z 方向的拖动，给传感器 z 方向的力，末端就会朝着相应的方向移动。

例 3 中，直线运动到 p1，在运动过程中开启 z 方向的力控且期望力为 6

例 1

```
ForcePositionHybridControl --force_direction={1,1,1,1,1,1} --damping_gain={0.03,0.03,0.03,0.3,0.3,0.3}
```

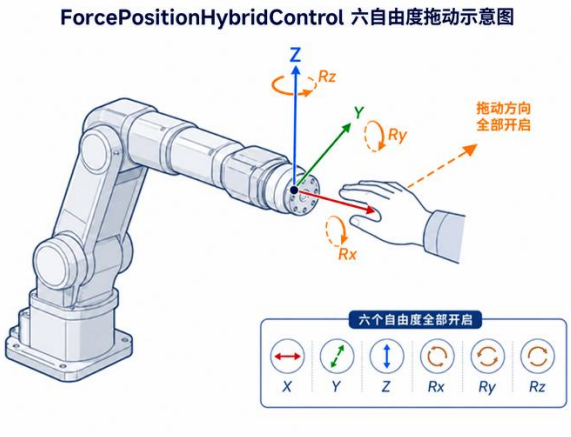
例 2

```
ForcePositionHybridControl --force_direction={0,0,1,0,0,0} --damping_gain={0.03,0.03,0.03,0.3,0.3,0.5} --force_target={0,0,5,0,0,0}
//其它参数可省略采用默认
```

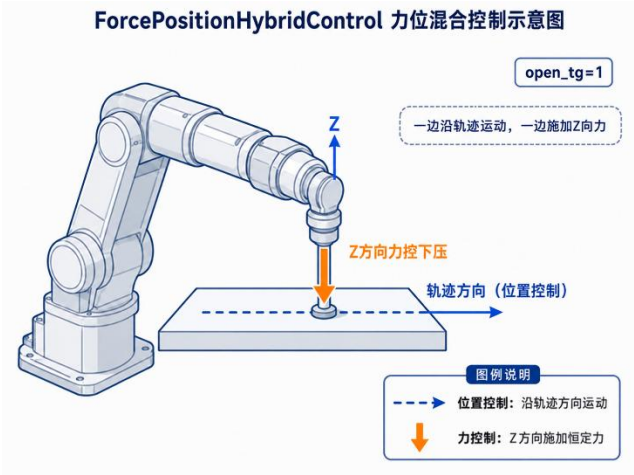
例 3

```
Var --type=robottarget --name=p1 --value={0.490,0.1,0.680,0,0,0.70710678,0,0.70710678}

MoveBlend --type=first_insert
MoveBlend --type=insert_line --robottarget=p1
ForcePositionHybridControl --force_target={0,0,6,0,0,0} --force_direction={0,0,1,0,0,0} --open_tg=1
```


ForcePositionHybridControl 六自由度拖动示意图
Z
Rz
Y
Ry
Rx
拖动方向
全部开启
六个自由度全部开启
X
Y
Z
Rx
Ry
Rz


ForcePositionHybridControl 单轴力控示意图（Z方向）
仅Z方向开启
force_direction={0,0,1,0,0,0}
force_target={0,0.5,0,0,0}
接触面 / 工件
自由度状态
X 未开启
Y 未开启
Z 已开启
Rx 未开启
Ry 未开启
Rz 未开启


ForcePositionHybridControl 力位混合控制示意图
open_tg=1
一边沿轨迹运动，一边施加Z向力
Z
Z方向力控下压
轨迹方向（位置控制）
图例说明
位置控制：沿轨迹方向运动
力控制：Z方向施加恒定力

3.DragInCST 电流环下机器人拖动控制

➢ **指令用法**

机器人根据关节电流、关节力矩或外部拖动力做出运动响应，实现机器人拖动示教功能。

➢ **指令参数**

	数据类型	含义
--cf_coef	Matrix	库伦摩擦补偿系数
--vf_coef	Matrix	粘性摩擦补偿系数
--zero_check	Matrix	零点判断阈值
--vel_limit	Matrix	拖动过程中的速度限制
--gravity_coef	Matrix	重力补偿系数
--ping_pong	Number	内部往返检测或周期参数
--ping_pong_amp	Number	往返检测或周期调节的幅值参数
--cutoff_freq	Number	电流、力矩或速度信号的一阶低通滤波器截止频率，用于降低信号噪声
--cbfSwitch	Matrix	安全约束开关
--joint_kp	Matrix	关节位置刚度参数
--joint_kd	Matrix	关节阻尼参数

➢ **指令说明**

拖动为电流环拖动模式，不依赖外部传感器，仅依据电机自身电流情况运行

在开始拖动前需要先设置好机器人重力参数以及动力学辨识

➢ **指令示例**

开启机器人拖动，摩擦系数均为 0，最大速度为 0.3m/s

```
DragInCST --cf_coef={0,0,0,0,0,0,0} --vf_coef={0,0,0,0,0,0,0} --vel_limit={0.3,0.3,0.3,0.3,0.3,0.3,0.3} --gravity_coef={1.0}
```

七、IO 模块指令

1.GetDI 读取输入电平状态

➢ **指令用法**　读取 IO 模块的指定数字输入（DI）通道的实时电平状态

➢ **指令参数**

	数据类型	含义
--di_name	String	设备通道号

➢ **指令说明**　该指令仅返回当前 DI 引脚的电平状态不改变硬件状态

➢ **指令示例**

例 1

```
GetDI --di_name=DI0; //读取数字通道 DO0 上的电平状态
```

2.SetDO 设置输出电平状态

➢ **指令用法**　设置指定数字输出（DO）通道的实时电平状态

➢ **指令参数**

	数据类型	含义
--do_name	String	设备通道号
--do_value	Number	目标输出电平逻辑状态

➢ **指令说明**　该指令为同步写入操作，执行后 DO 引脚电平即刻变化

➢ **指令示例**

例 1

```
SetDO --do_name=DO0 --do_value=1; //设置数字通道 DO0 上的电平状态为 1 状态
```

3.DOPulse 输出数字脉冲信号

➢ **指令用法**	设置指定数字输出（DO）通道输出数字脉冲信号

➢ **指令参数**	数据类型	含义
--do_name	String	设备通道号
--pulse_active	Number	是否启用脉冲输出
--high_cycles	Number	高电平持续周期数
--low_cycles	Number	低电平持续周期数

➢ **指令说明**	--do_name 必须是已定义的有效 DO 通道名

➢ **指令示例**

例 1

DOPulse --do_name=DO0 --pulse_active=1 --high_cycles=10 --low_cycles=10; //数字输出通道 DO0 上输出一个由高电平和低电平组成的、可配置时长的脉冲序列

4.AddIO 注册新的 I/O 信号输出口

➢ **指令用法**	注册指定数字输入输出（DI/O）通道口

➢ **指令参数**	数据类型	含义
--io_type	String	指定要添加的 I/O 信号类型
--io_num	String	物理通道编号
--io_name	String	变量名

➢ **指令说明**	--io_num 是硬件层地址； --io_name 是软件层标识符

➢ **指令示例**

例 1

AddIO --io_type=DO --io_num=1 --io_name=EMERGENCY_STOP;//将物理 DO1 通道注册为变量 EMERGENCY_STOP

八、底盘导航指令

1.ModbusTCP 运动指令

➢ **指令用法**

基于 Modbus TCP 协议，控制小车执行笛卡尔空间下的运动控制，支持点位运动和自动导航站点运动两种模式，可设置目标位姿、运动速度、导航模式及目标站点，实现手动控制或自动导航。

➢ **指令参数**

指令参数	数据类型	含义
--x_axis	Number	目标 X 轴方向移动
--y_axis	Number	目标 Y 轴方向移动
--theta	Number	目标姿态角度（弧度 / 角度，默认 0）
--vx	Number	X 方向线速度系数，值越大，X 向运动越快（默认 0.2）
--vy	Number	Y 方向线速度系数，值越大，Y 向运动越快（默认 0.2）
--vtheta	Number	旋转角速度系数，值越大，旋转越快（默认 0.2）
--navigate_open	Number	导航模式开关：0= 手动控制（直接控制位姿），1= 自动导航（前往目标站点）（默认 0）
--target_station_id	Number	自动导航模式下的目标站点编号（默认 1）

➢ **指令说明**

1. 手动模式（navigate_open=0）下，以 x_axis/y_axis/theta 为目标位姿，按 vx/vy/vtheta 设置的速度执行运动。

2. 自动导航模式（navigate_open=1）下，将忽略位姿与速度参数，直接前往 target_station_id 指定的站点。

3. 切换导航模式时，需确保站点地图已配置完成，否则导航可能失败。

➢ **指令示例**

例 1

CarModbusTCPJGBOT --navigate_open=0 --x_axis=1 --y_axis=1 --theta=0 --vx=0.1 --vy=0.1 --vtheta=0.3// 手动控制模式：直接移动到目标位姿

CarModbusTCPJGBOT --navigate_open=1 --target_station_id=2// 自动导航模式：前往指定站点